LAB REPORT 8

Performance Evaluation: Throughput, Jitter, and Loss vs. Distance

Student Name	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering International Program, College of Computing, PSU Phuket
Course	977-337 Network Programming and Simulation
Week	8
Environment	Ubuntu 24.04 Linux VM, Mininet-WiFi Python API (`mn_wifi`), Open vSwitch AP, iperf3 3.16, Python 3, matplotlib 3.11.1
Source files	`lab08_performance/measure.py`, `plot_results.py`, `results.json`, `performance_plot.png` in the netprog-2026 repository

1. Objective

This lab measures the four core performance metrics — throughput, delay, jitter and packet loss — between two Mininet-WiFi stations using iperf3 in UDP mode. I first ran one manual iperf3 UDP test to locate the fields inside the JSON output, wrapped the test in a reusable Python function, then automated a sweep that moves sta1 from 5 m to 30 m away from the access point (whose `range=30`) and saves the results to `results.json`. Finally I plotted throughput and loss against distance with matplotlib and interpreted the trend.

2. Topology and Measurement Setup

Node / setting	Value	Role
ap1	position 0,50,0 · SSID netprog-ap · mode g · ch 1 (2412 MHz) · `range=30`	single access point, `failMode='standalone'`
sta2	position 0,50,0 (10.0.0.2)	iperf3 server, fixed at the AP
sta1	5 → 10 → 15 → 20 → 25 → 30 m (`setPosition`)	iperf3 client, moved away from ap1
Test	`iperf3 -c 10.0.0.2 -u -b 10M -t 5 -J`	UDP, 10 Mbit/s target, 5 s per distance
Distances vs. range	5 / 10 / 15 / 20 / 25 / 30 m = 17 / 33 / 50 / 67 / 83 / **100 %** of range	30 m is exactly the edge of ap1's coverage

Each distance takes about 9 s (1 s settle + 1 s server start + 5 s test + 1 s wait + pkill), so the whole sweep runs in roughly one minute, as the lab sheet expects.

3. Final Scripts and Changes Needed in This Environment

The skeleton code from the lab sheet was kept (TODO 1–4 filled in exactly as given), with the following additions for this installation:

#	Problem observed	Cause	Fix
1	(Known from Lab 7) traffic through an OVS access point is dropped when no controller runs	A script using `Mininet_wifi()` starts no controller, so OVS has no flow rules	`failMode='standalone'` on ap1; ping sta1 → sta2 was 0% loss before testing (Figure 2)
2	First plot: the throughput line was flat at 9.999 Mbps at every distance	In iperf3 3.16, `end.sum` has `"sender": true` — it is the rate that was *sent*, which is always the 10 Mbit/s target	Also store `end.sum_received` as `received_mbps` (rate really delivered) and plot both lines
3	First plot: the loss panel looked empty (all points on the x-axis, axis 0–0.05 %)	Loss was 0 % at every distance, so matplotlib auto-scaled to a tiny range	Fixed y-axis 0–5 %, value labels "0%", and jitter on a second y-axis
4	Risk of a crash or hang if sta1 loses the AP at 30 m (no `end.sum` in JSON)	iperf3 writes only `{"error": ...}` when it cannot connect	`pkill` old servers, `timeout` on the client, and record an error entry instead of a `KeyError` (not triggered in this run)

I also log the station's association (`wintfs[0].associatedTo`) and the signal from `iw dev sta1-wlan0 link` at each distance as extra evidence. Final `measure.py` (Parts C and D; `--single 5` runs one measurement for Task 2):

```
import json
import sys
import time

from mn_wifi.net import Mininet_wifi
from mn_wifi.cli import CLI
from mininet.log import setLogLevel, info

DISTANCES = [5, 10, 15, 20, 25, 30]


def run_iperf3_udp(server_sta, client_sta, server_ip, duration=5, bitrate="10M"):
    # Free port 5201 in case an iperf3 server is left over from a failed run
    server_sta.cmd("pkill -9 iperf3")
    server_sta.cmd(f"iperf3 -s -1 -J > /tmp/server_{client_sta.name}.json &")
    time.sleep(1)  # give the server a moment to start listening
    # TODO 1 (timeout stops the client hanging if the station has lost the AP)
    client_sta.cmd(f"timeout {duration + 15} iperf3 -c {server_ip} -u -b {bitrate} "
                   f"-t {duration} -J > /tmp/client_{client_sta.name}.json")
    time.sleep(1)
    try:
        with open(f"/tmp/client_{client_sta.name}.json") as f:
            data = json.load(f)
    except (OSError, json.JSONDecodeError) as e:
```

```
        data = {"error": f"no valid JSON from client ({e.__class__.__name__})"}

    if "error" in data or "sum" not in data.get("end", {}):
        # Client could not reach the server: record a total outage instead of crashing
        return {"throughput_mbps": 0.0, "jitter_ms": None, "loss_percent": 100.0,
                "error": data.get("error", "no end.sum in iperf3 JSON")}

    # TODO 2
    summary = data["end"]["sum"]
    result = {
        "throughput_mbps": round(summary["bits_per_second"] / 1e6, 3),
        "jitter_ms": round(summary["jitter_ms"], 3),
        "loss_percent": round(summary["lost_percent"], 3),
    }
    # end.sum is the sender's rate; iperf3 3.16 also reports what really arrived
    received = data["end"].get("sum_received")
    if received:
        result["received_mbps"] = round(received["bits_per_second"] / 1e6, 3)
    return result
```

```
def link_info(sta):
    """Return (associated AP interface, signal in dBm) for the station."""
    intf = sta.wintfs[0]
    assoc = sta.params.get('associatedTo') or intf.associatedTo
    signal = None
    for line in sta.cmd(f"iw dev {intf.name} link").splitlines():
        if "signal:" in line:
            signal = int(line.split()[1])
    return (str(assoc) if assoc else None), signal


def topology(single_distance=None):
    net = Mininet_wifi()

    info("*** Creating nodes\n")
    sta1 = net.addStation('sta1', position='5,50,0')
    sta2 = net.addStation('sta2', position='0,50,0')
    net.addAccessPoint('ap1', ssid='netprog-ap', mode='g', channel='1',
                       position='0,50,0', range=30, failMode='standalone')

    info("*** Configuring wifi nodes\n")
    net.configureWifiNodes()
```

```
    info("*** Starting network\n")
    net.build()
    net.start()
    time.sleep(2)  # let both stations finish associating

    distances = [single_distance] if single_distance is not None else DISTANCES
    results = []
    for d in distances:
        info(f"*** Measuring at distance {d}m\n")
        # TODO 3
        sta1.setPosition(f"{d},50,0")
        time.sleep(1)  # let the wireless link settle after moving
        assoc, signal = link_info(sta1)
        r = run_iperf3_udp(sta2, sta1, sta2.IP())
        # TODO 4
        r["distance_m"] = d
        r["associated_to"] = assoc
        r["signal_dbm"] = signal
        results.append(r)
        print(d, r)
```

```
    if single_distance is None:
        with open("results.json", "w") as f:
            json.dump(results, f, indent=2)
        info("*** Saved results.json\n")

    CLI(net)
    net.stop()


if __name__ == "__main__":
    setLogLevel("info")
    if len(sys.argv) == 3 and sys.argv[1] == "--single":
        topology(single_distance=int(sys.argv[2]))   # Task 2: one measurement
    else:
        topology()                                    # Task 3: full sweep
```

Final `plot_results.py` (Part E):

```
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

AP_RANGE = 30  # ap1 range configured in measure.py

with open("results.json") as f:
    results = json.load(f)

distances = [r["distance_m"] for r in results]
# end.sum in iperf3 3.16 is the SENDER's rate; sum_received is what actually arrived
throughput = [r["throughput_mbps"] for r in results]
received = [r.get("received_mbps", r["throughput_mbps"]) for r in results]
loss = [r["loss_percent"] for r in results]
jitter = [r["jitter_ms"] or 0 for r in results]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.3))
```

```
# Panel 1: throughput vs distance
ax1.plot(distances, received, marker="o", color="tab:blue", label="Received (delivered)")
ax1.plot(distances, throughput, marker="s", linestyle="--", color="grey",
         label="Sent (iperf3 end.sum)")
for x, y in zip(distances, received):
    ax1.annotate(f"{y:.2f}", (x, y), textcoords="offset points",
                 xytext=(-14, -12) if x == AP_RANGE else (0, -15),
                 ha="center", fontsize=8, color="tab:blue")
ax1.set_ylim(0, 11)
ax1.set_xlabel("Distance from AP (m)")
ax1.set_ylabel("Throughput (Mbps)")
ax1.set_title("Throughput vs. Distance")
ax1.legend(loc="lower left", fontsize=8)
```

```
# Panel 2: packet loss vs distance (jitter on a second axis)
l1 = ax2.plot(distances, loss, marker="o", color="crimson", label="Packet loss (%)")
for x, y in zip(distances, loss):
    ax2.annotate(f"{y:g}%", (x, y), textcoords="offset points", xytext=(0, 7),
                 ha="center", fontsize=8, color="crimson")
ax2.set_ylim(0, 5)
ax2.set_xlabel("Distance from AP (m)")
ax2.set_ylabel("Packet Loss (%)")
ax2.set_title("Packet Loss (and Jitter) vs. Distance")
ax2b = ax2.twinx()
l2 = ax2b.plot(distances, jitter, marker="^", color="darkorange", label="Jitter (ms)")
ax2b.set_ylim(0, max(jitter) * 1.6 if max(jitter) > 0 else 1)
ax2b.set_ylabel("Jitter (ms)")
ax2.legend(l1 + l2, [l.get_label() for l in l1 + l2], loc="upper left", fontsize=8)

for ax in (ax1, ax2):
    ax.set_xticks(distances)
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.axvline(AP_RANGE, color="grey", linestyle="--", linewidth=1)
    ax.text(AP_RANGE - 0.5, ax.get_ylim()[1] * 0.45, "AP range = 30 m", rotation=90,
            color="grey", va="center", ha="right", fontsize=8)
```

```
fig.tight_layout()
fig.savefig("performance_plot.png", dpi=150)
print("Saved performance_plot.png")
```

4. Lab Tasks and Results

Part A — Project Setup

```
diz@diz:~/netprog-2026/lab08_performance$ iperf3 --version
iperf 3.16 (cJSON 1.7.15)
Linux diz 7.0.0-38-generic #38~24.04.4-Ubuntu SMP PREEMPT_DYNAMIC Mon Sep 14 16:37:11 UTC 2 x86_64
Optional features available: CPU affinity setting, IPv6 flow label, SCTP, TCP congestion algorithm s
etting, sendfile / zerocopy, socket pacing, authentication, bind to device, support IPv4 don't fragm
ent, POSIX threads
diz@diz:~/netprog-2026/lab08_performance$ source ~/netprog-env/bin/activate
(netprog-env) diz@diz:~/netprog-2026/lab08_performance$
(netprog-env) diz@diz:~/netprog-2026/lab08_performance$ python3 -c "import matplotlib; print(matplot
lib.__version__)"
3.11.1
```

Figure 1: iperf3 3.16 (supports -J) and matplotlib 3.11.1 inside netprog-env

Task 1 — Manual iperf3 UDP Baseline (Part B)

```
*** Starting CLI:
mininet-wifi> sta1 ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.877 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.548 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.645 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2063ms
rtt min/avg/max/mdev = 0.548/0.690/0.877/0.138 ms
mininet-wifi>
mininet-wifi> sta2 iperf3 -s -1 -J > /tmp/server.json &
mininet-wifi>
mininet-wifi> sta1 iperf3 -c 10.0.0.2 -u -b 10M -t 5 -J > /tmp/client.json
mininet-wifi>
mininet-wifi> sta1 tail -n 45 /tmp/client.json
                        "start":        0,
                        "end":  5.000848,
                        "seconds":      5.000848,
                        "bytes":        6249568,
                        "bits_per_second":      9998652.8883705232,
                        "jitter_ms":    0.23973663810558177,
                        "lost_packets": 0,
                        "packets":      4316,
                        "lost_percent": 0,
                        "sender":       true
                },
                "sum_sent":     {
                        "start":        0,
                        "end":  5.000328,
                        "seconds":      5.000328,
                        "bytes":        6249568,
                        "bits_per_second":      9998652.8883705232,
                        "jitter_ms":    0,
                        "lost_packets": 0,
                        "packets":      4316,
                        "lost_percent": 0,
                        "sender":       true
                },
                "sum_received": {
                        "start":        0,
                        "end":  5.000848,
                        "seconds":      5.000848,
                        "bytes":        6249568,
                        "bits_per_second":      9997613.2048004642,
                        "jitter_ms":    0.23973663810558177,
                        "lost_packets": 0,
                        "packets":      4316,
                        "lost_percent": 0,
                        "sender":       false
                },
                "cpu_utilization_percent":      {
                        "host_total":   100.751391942278,
                        "host_user":    100.71026989616334,
                        "host_system":  0.041102054842855611,
                        "remote_total": 5.0391440401896537,
                        "remote_user":  1.1680523202974511,
                        "remote_system":        3.871091719892203
                }
        }
}
```

Figure 2: Part B at 5 m: ping check, manual iperf3 UDP test, and the end of the client JSON (end.sum, end.sum_sent, end.sum_received)

```
mininet-wifi>
mininet-wifi> sta1 python3 -c "import json; d=json.load(open('/tmp/client.json')); print(json.dumps(
d['end']['sum'], indent=2))"
{
  "start": 0,
  "end": 5.000848,
  "seconds": 5.000848,
  "bytes": 6249568,
  "bits_per_second": 9998652.888370523,
  "jitter_ms": 0.23973663810558177,
  "lost_packets": 0,
  "packets": 4316,
  "lost_percent": 0,
  "sender": true
}
```

Figure 3: data["end"]["sum"] printed by Python: bits_per_second, jitter_ms and lost_percent

Field in data["end"]["sum"]	Value	Meaning
`bits_per_second`	9,998,652.9 (≈ **10.00 Mbps**)	throughput (sender side, `"sender": true`)
`jitter_ms`	**0.240 ms**	variation in packet spacing measured by the server
`lost_percent`	**0 %** (0 of 4316 packets)	no datagram was lost
`sum_received.bits_per_second`	9,997,613.2 (≈ 10.00 Mbps)	rate that actually arrived at sta2

The JSON is valid and readable. The three required fields sit inside `["end"]["sum"]`, which is exactly the path the Python function parses. At 5 m the link carries the full 10 Mbit/s target with 0 % loss and only 0.24 ms jitter. The ping before the test averaged 0.690 ms RTT, i.e. a one-way delay of about 0.35 ms.

Task 2 — run_iperf3_udp() for a Single Distance (Part C)

```
*** Creating nodes
*** Configuring wifi nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 30.00m for requested 30.00m (delta +0.00m)
*** Starting network
*** Starting controller(s)
*** Starting L2 nodes
ap1 ...
*** Measuring at distance 5m
5 {'throughput_mbps': 9.998, 'jitter_ms': 0.236, 'loss_percent': 0, 'received_mbps': 9.998, 'distanc
e_m': 5, 'associated_to': 'ap1-wlan1', 'signal_dbm': None}
```

Figure 4: sudo python3 measure.py --single 5: one call of run_iperf3_udp() returns a result dictionary

One call of `run_iperf3_udp(sta2, sta1, sta2.IP())` at 5 m returned `throughput_mbps: 9.998`, `jitter_ms: 0.236`, `loss_percent: 0` — the same values as the manual test in Task 1, so the function parses the JSON correctly. sta1 is associated to `ap1-wlan1`. `signal_dbm` is `None` here because `iw` was queried only 1 s after the network started, before it reported a signal value (see Reflection Q2).

Task 3 — Full Distance Sweep and results.json (Part D)

```
*** Creating nodes
*** Configuring wifi nodes
*** ap1-wlan1: minimum tx power (1 dBm) yields 30.00m for requested 30.00m (delta +0.00m)
*** Starting network
*** Starting controller(s)
*** Starting L2 nodes
ap1 ...
*** Measuring at distance 5m
5 {'throughput_mbps': 9.999, 'jitter_ms': 0.236, 'loss_percent': 0, 'received_mbps': 9.998, 'distanc
e_m': 5, 'associated_to': 'ap1-wlan1', 'signal_dbm': None}
*** Measuring at distance 10m
10 {'throughput_mbps': 9.998, 'jitter_ms': 0.295, 'loss_percent': 0, 'received_mbps': 9.675, 'distan
ce_m': 10, 'associated_to': 'ap1-wlan1', 'signal_dbm': -49}
*** Measuring at distance 15m
15 {'throughput_mbps': 9.999, 'jitter_ms': 0.336, 'loss_percent': 0, 'received_mbps': 9.205, 'distan
ce_m': 15, 'associated_to': 'ap1-wlan1', 'signal_dbm': -49}
*** Measuring at distance 20m
20 {'throughput_mbps': 9.999, 'jitter_ms': 0.379, 'loss_percent': 0, 'received_mbps': 8.758, 'distan
ce_m': 20, 'associated_to': 'ap1-wlan1', 'signal_dbm': -49}
*** Measuring at distance 25m
25 {'throughput_mbps': 9.999, 'jitter_ms': 0.423, 'loss_percent': 0, 'received_mbps': 8.331, 'distan
ce_m': 25, 'associated_to': 'ap1-wlan1', 'signal_dbm': -49}
*** Measuring at distance 30m
30 {'throughput_mbps': 9.999, 'jitter_ms': 0.471, 'loss_percent': 0, 'received_mbps': 7.928, 'distan
ce_m': 30, 'associated_to': 'ap1-wlan1', 'signal_dbm': -49}
*** Saved results.json
```

Figure 5: sudo python3 measure.py: all six distances measured and results.json saved

Distance	% of range	Sent (Mbps)	Received (Mbps)	Change vs 5 m	Jitter (ms)	Loss (%)
5 m	17 %	9.999	9.998	—	0.236	0
10 m	33 %	9.998	9.675	−3.2 %	0.295	0
15 m	50 %	9.999	9.205	−7.9 %	0.336	0
20 m	67 %	9.999	8.758	−12.4 %	0.379	0
25 m	83 %	9.999	8.331	−16.7 %	0.423	0
30 m	**100 %**	9.999	**7.928**	**−20.7 %**	**0.471**	**0**

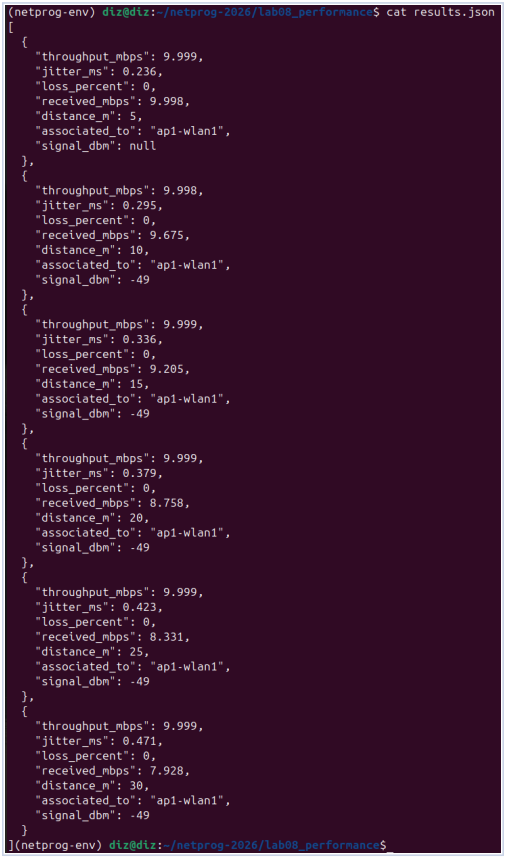


Figure 6: Contents of results.json — six entries, one per distance

The sweep completed and `results.json` contains six complete entries. sta1 stayed associated to `ap1-wlan1` at every distance, including 30 m. To confirm the state at the edge of the range, I checked the link after the sweep while sta1 was still at 30 m:

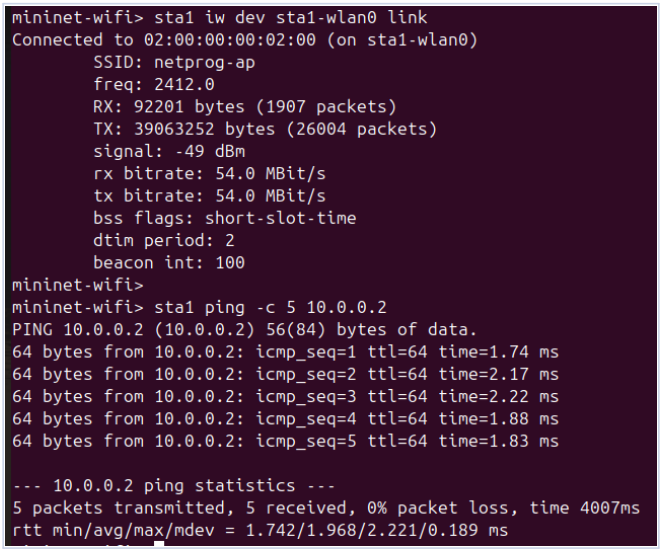


Figure 7: At 30 m: sta1 still connected to netprog-ap; ping 0 % loss, average RTT 1.968 ms

At 30 m the ping RTT averaged **1.968 ms** compared with 0.690 ms at 5 m in Part B, so the estimated one-way delay (RTT / 2) grew from about 0.35 ms to about 0.98 ms — almost three times higher — while the link stayed up. `iw` reported the same signal (−49 dBm) at every distance, which means this Mininet-WiFi setup does not model distance in the radio signal itself (see Task 6).

Task 4 — Throughput vs. Distance (Part E)

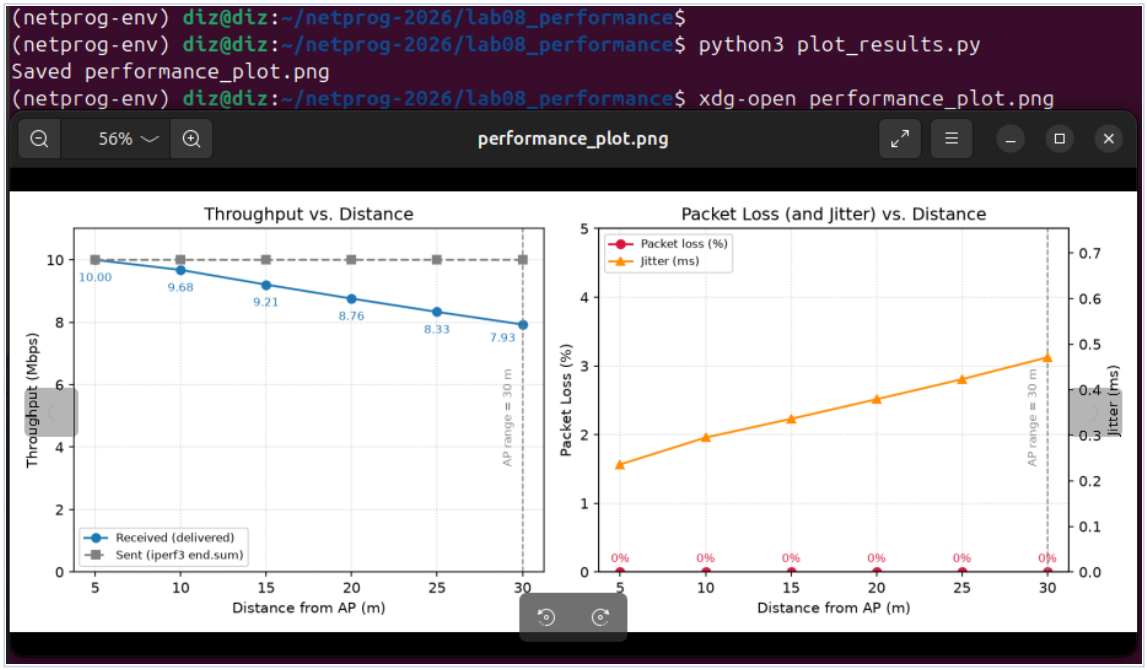


Figure 8: python3 plot_results.py run in netprog-env: performance_plot.png saved and opened on the VM

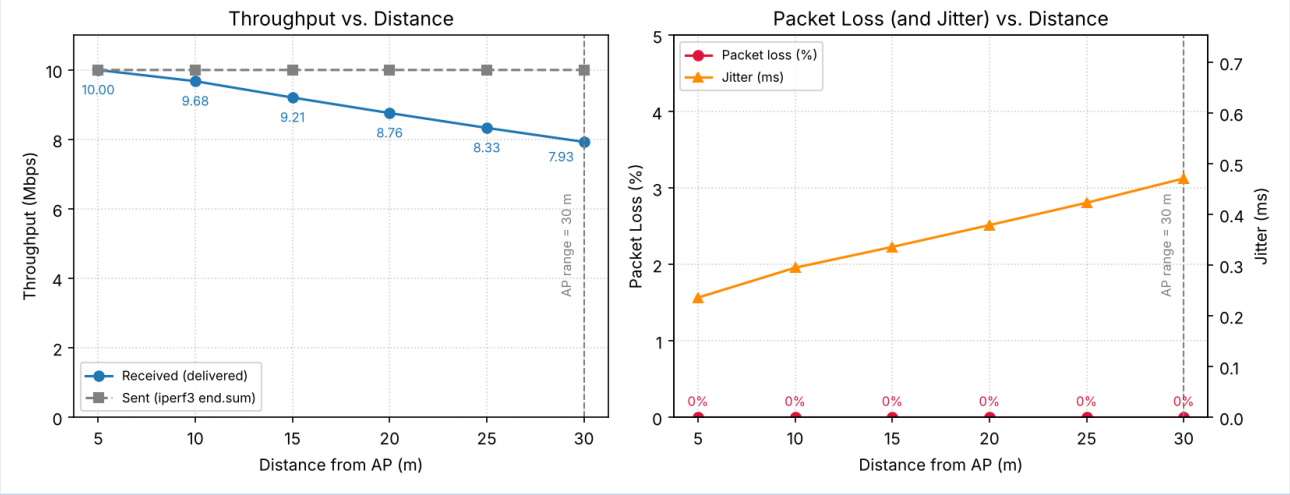


Figure 9: performance_plot.png (full resolution): delivered vs. sent throughput (left) and packet loss with jitter (right); dashed line = AP range 30 m

The rate that iperf3 *sent* (`end.sum`) stays at 9.999 Mbps at every distance, because UDP always transmits at the configured `-b 10M` — so the grey "Sent" line is flat and is shown only for comparison. The rate actually **delivered** to sta2 (`sum_received`) shows the real trend: it falls almost linearly from **9.998 Mbps at 5 m to 7.928 Mbps at 30 m**, about 0.4–0.47 Mbps per 5 m (≈0.083 Mbps per metre, −20.7 % overall). The drop starts already at 10 m (−3.2 %) and continues steadily; there is no sharp "cliff" before 30 m, because 30 m is still inside (exactly at) ap1's configured range, so sta1 never disconnected.

Task 5 — Loss vs. Distance

Packet loss stayed at **0 % at all six distances**, including 30 m = range, so loss did not increase gradually — it stayed at 0 % right up to the edge of the range. Instead, the degradation appeared as lower delivered throughput and higher jitter (0.236 → 0.471 ms, doubled) and delay (RTT 0.69 → 1.97 ms). Every one of the 6,249,568 bytes still arrived, but over a longer time (about 6.3 s instead of 5 s at 30 m): the packets were queued and delivered more slowly rather than dropped. Relating this to Lab 6, sta1 was in the **"degraded"** state from about 10 m onwards (still associated, but slower and more variable), not yet in the "disconnected" state; loss would be expected to jump to 100 % only once sta1 moves beyond 30 m and loses the AP.

Task 6 — Why Performance Degrades, and Model Limitations

Mininet-WiFi computes each station's link quality from its distance to the AP with a propagation model: the further sta1 is, the weaker the modelled signal, so the emulator gives the link a lower data rate and more delay until, at the configured range, the station is at the edge of association. In this setup the model is applied by shaping the link rather than by dropping frames, which is why I saw a smooth, deterministic loss-free decline. Real radio propagation is different: walls and obstacles, multipath fading and interference from other networks make the signal fluctuate randomly, so a real station would see Wi-Fi rate adaptation, MAC retransmissions and some packet loss well before the nominal range — and the results would vary between runs, unlike the perfectly linear values here (and the constant −49 dBm that `iw` reported).

5. Verification Checklist

#	Checkpoint	Evidence	✓
1	Manual iperf3 UDP baseline (Part B) produces valid, readable JSON output	Figures 2–3	✓
2	run_iperf3_udp() (Part C) returns a correct dictionary for a single measurement	Figure 4	✓
3	Full distance sweep (Part D) completes and produces a 6-entry results.json	Figures 5–6	✓
4	performance_plot.png (Part E) is generated with both panels correctly labeled (after fixing the flat/empty first version)	Figure 9	✓
5	Throughput and loss trends are correctly described relative to range=30 (Tasks 4–5): delivered throughput −20.7 % by 30 m, loss 0 % up to the range	Tasks 4–5	✓

6. Reflection Questions

Q1. Why does this lab use iperf3's UDP mode (-u) rather than TCP mode to measure jitter and packet loss? What would you have to do differently with TCP to estimate similar metrics?

UDP sends at a fixed rate and never retransmits, so the server can count exactly which datagrams were missing (`lost_percent`) and how unevenly they arrived (`jitter_ms`). TCP hides both: lost segments are retransmitted and congestion control slows the sender down, so loss turns into lower throughput and iperf3 reports no jitter at all. With TCP I would have to estimate them indirectly — loss from the `retransmits` counter in iperf3's JSON (or `ss -ti`), and delay/jitter from a ping running alongside (RTT / 2 and its `mdev`) or from packet timestamps captured with tcpdump.

Q2. Why is a short pause (time.sleep) necessary after repositioning a station and before running the next performance measurement?

`setPosition()` only changes the coordinates; Mininet-WiFi then has to recompute the distance-based link parameters (and, if needed, the association) for the new position. Without a pause the first part of the 5 s test could still run on the old link settings, mixing two distances in one result. My own data shows that the wireless

state needs time to settle: the very first reading at 5 m, taken shortly after start-up, returned `signal_dbm: null` because `iw` had not reported a value yet, while all later readings (taken after the 1 s settle) did.

Q3. At what distance does performance begin to degrade noticeably, and how does this relate to the range=30 parameter configured on the AP?

Degradation starts gradually from about **10 m** (delivered throughput −3.2 %, jitter 0.236 → 0.295 ms) and becomes clearly noticeable from **15–20 m** (−7.9 % to −12.4 %, i.e. from half of the range onwards), reaching −20.7 % and doubled jitter at 30 m. So `range=30` does not mark where degradation starts; it marks the edge of association — performance scales down smoothly as distance approaches the range, and the station is still connected (0 % loss) exactly at 30 m. Only beyond the range would the link be lost completely.

7. Conclusion

iperf3 in UDP mode with `-J` made all metrics directly readable from `data["end"]["sum"]`, and the automated sweep produced a complete six-entry `results.json`. The key finding is that iperf3 3.16's `end.sum` reports the *sent* rate, so the delivered rate (`sum_received`) was needed to see the trend: delivered throughput fell linearly from 9.998 to 7.928 Mbps (−20.7 %) as sta1 moved from 5 m to 30 m, jitter doubled (0.236 → 0.471 ms) and RTT rose from 0.69 to 1.97 ms, while loss stayed at 0 % and sta1 stayed associated up to the configured `range=30` — a smooth, deterministic trend from Mininet-WiFi's simplified distance model.